

SAUCEDEMO

Test Automation — Test Plan

Selenium WebDriver · Pytest · Page Object Model (POM) · Python · uv

Project Name: SauceDemo POM Automation Framework

Application Under Test: SauceDemo E-Commerce Web Application

Application URL: <https://www.saucedemo.com/>

Repository: saucedemo_pom

Test Type: Functional Regression Testing (UI Automation)

Prepared By: Nirjala K.C.

Role: QA Automation Engineer

Tools & Frameworks: Python, Selenium WebDriver, Pytest, Page Object Model (POM), uv

Document Status: Approved

1. Objective

The primary objective of this project is to build a reliable automation suite that verifies the core ecommerce workflows of the SauceDemo application. Utilizing **Selenium WebDriver**, **Pytest**, and the **Page Object Model (POM)** design pattern, this framework automates critical user journeys— such as authentication, product management, and the checkout funnel—to catch regressions early in the development cycle. In addition to standard end-to-end user flows, the suite tests system constraints and boundary conditions. It explicitly validates how the application handles input failures (e.g., missing fields), security boundaries (e.g., direct URL manipulation), and state inconsistencies (e.g., checking out with an empty cart).

Ultimately, this project delivers a clean, low-maintenance test suite that runs fast and scales easily. It provides high-signal feedback to the team, ensuring that a passing test run guarantees application stability and a smooth user experience.

2. Scope

To keep the suite focused and maintainable, this cycle draws a clear line around what is being tested and what isn't.

In Scope

The suite covers the complete front-end user journey on the application. This includes authentication validation (valid, invalid, empty-field, and case-sensitive inputs), product catalog operations (browsing and sorting by price/name), cart modifications (adding/removing items), and the complete checkout pipeline (form submission, tax/total calculations, and order confirmation). Critical functional edge cases—such as invalid zip codes, checking out with an empty cart, and bypassing steps via direct URL manipulation—are prioritized as core test cases.

Out of Scope

This cycle is strictly limited to client-side browser behavior; backend API, database validation, and security penetration testing are excluded. Furthermore, mobile responsive rendering and live payment gateway processing are not covered. Test execution is constrained entirely to functional validation on Google Chrome; cross-browser, load, and performance testing are outside the scope of this framework.

3. Test Approach

This automation framework focuses on client-side functional validation, using **Selenium WebDriver** to drive Chrome and **Pytest** to manage test execution. We use the **Page Object Model (POM)** design pattern to cleanly separate the test logic from the UI elements. Tests focus purely on asserting business logic, while page objects encapsulate page locators and user actions. This separation ensures that if a locator changes, it only needs an update in one central place.

To prevent flaky runs, the framework avoids hardcoded sleep times, handling elements synchronization entirely through **explicit waits** that poll for state changes dynamically. Test independence is strictly enforced; each test kicks off in an isolated browser instance to prevent cascading failures. All test run summaries and failures are captured cleanly in an HTML report for immediate debugging, with dependencies managed seamlessly via uv.

When a test fails, the breakdown is treated as a high-signal defect rather than background noise. Any assertion failure or unexpected application state is logged systematically with clear reproduction steps and severity levels, keeping our test case tracking and active bug reports tightly synchronized with the codebase.

4. Test Environment

All automation tests run against the live SauceDemo web application inside a **Google Chrome** browser on a **Windows** host machine. The runtime environment and framework ecosystem use the following specified toolchain configuration:

Parameter	Details
Application	SauceDemo
URL	https://www.saucedemo.com/
Browser	Google Chrome
Automation Tool	Selenium WebDriver v4.45.0+
Framework	Pytest v9.1.1+
Language	Python 3.14
Package/Env Manager	uv
Design Pattern	Page Object Model (POM)
Failure Evidence	Screenshot captured on failure

5. Test Architecture

The automation suite is built using the Page Object Model (POM) architecture, separating page interactions from test logic.

```
saucedemo_pom/
├── docs/
│   ├── bugreport_saucedemo.pdf
│   └── test_case_saucedemo.xlsx
├── pages/
│   ├── base_page.py
│   ├── login_page.py
│   ├── inventory_page.py
│   ├── cart_page.py
│   ├── checkout_step_one_page.py
│   ├── checkout_step_two_page.py
│   └── checkout_complete_page.py
├── tests/
│   ├── test_login.py
│   ├── test_inventory.py
│   ├── test_cart.py
│   ├── test_checkout_step_one.py
│   ├── test_checkout_step_two.py
│   ├── test_checkout_complete.py
│   └── test_end_to_end.py
├── conftest.py
├── pyproject.toml
├── uv.lock
└── README.md
```

Design Principles

Every test manages its own driver session through a Pytest fixture, so tests stay independent and one failure can't leave a stale browser state behind for the next test. Waiting logic is centralized in `base_page.py` — the `find`, `find_visible`, `click` and `type` helper methods all use `WebDriverWait` under the hood — so no test file needs to reach for a hardcoded sleep to deal with a slow-loading element. With one page object per screen and shared helpers doing the heavy lifting, adding a test for a new page is mostly a matter of writing a small, focused page object and a handful of test functions that read almost like plain English.

6. Test Scope Coverage (Modules Under Testing)

The application breaks down naturally into the modules below, and the test suite mirrors that structure one-to-one — which makes it easy to tell, at a glance, how much of the application a given test run actually touched.

Module	Purpose
Authentication (Login)	Validate login with valid, invalid, empty and case-sensitive credentials
Inventory	Product sorting (price/name), footer/social links, reset app state behaviour
Cart	Add/remove items, product detail accuracy, checkout/continueshopping navigation
Checkout Step One	Shipping information form submission and field validation
Checkout Step Two	Order summary accuracy — item totals, tax and grand total
Checkout Complete	Order confirmation message and post-order state verification
End-to-End	Full purchase journey from login through order confirmation
Negative Testing	Invalid inputs, empty-state and direct-URL access edge cases

7. Test Execution Types

Not every test in the suite is doing the same job. Some confirm that a single piece of functionality behaves correctly in isolation, some walk through the full purchase journey end to end, and others exist specifically to try to break things. Grouping them this way makes it easier to reason about coverage rather than just counting how many tests exist

Type	Category	Description
Functional Testing	Core Testing	Validation of major application functionalities
Automation Testing	Automation	Automated execution using Selenium WebDriver
UI Testing	Interface	Validation of UI components and user interactions
End-to-End Testing	Workflow	Complete purchase-flow validation (login → order confirmation)
Negative Testing	Validation	Invalid inputs and failure/edge-case handling
Regression Testing	Stability	Re-verification of existing functionality across builds

8. Dynamic Content Handling

SauceDemo's UI updates asynchronously in places — a cart badge appearing, a button label flipping from "Add to cart" to "Remove", a page transitioning after a form submission — and timing issues here are a common source of flaky tests. Rather than pausing for a fixed number of seconds and hoping the page has caught up, the framework waits on specific conditions:

`presence_of_element_located` when an element just needs to exist, `visibility_of_element_located` when it needs to be seen, and `element_to_be_clickable` before any click is attempted. Navigation is confirmed by checking `driver.current_url`, which is a reliable way to know a transition genuinely completed rather than assuming it did.

9. User Actions Performed

Action	Purpose
Click Actions	Navigation, add/remove-to-cart and button interactions
Send Keys	Login form and checkout shipping-form input
Element Visibility Checks	Confirming inventory, cart and confirmation elements render
Dropdown Selection	Product sorting by price and name
URL / Navigation Assertions	Verifying correct page transitions and blocked/allowed direct access
Form Submission	Checkout and login validation

Test Data & Execution Strategy

Test data is chosen deliberately rather than arbitrarily: `standard_user` / `secret_sauce` covers the validlogin path, while `invalid`, `empty` and `mismatched-case` credentials exercise the negative paths. SauceDemo's other special accounts — `locked_out_user`, `problem_user` and similar — are intentionally left out of the login test data, since they represent a different kind of scenario that falls outside this cycle's scope rather than being an oversight.

Each test case runs in complete isolation, with its own WebDriver session created and torn down by a Pytest fixture, so nothing carries over between tests by accident. A screenshot on failure is recommended so a bug can be understood from the report alone, and every run produces an HTML summary via `pytest-html` that's easy to skim for a pass/fail overview.

Execution Flow

The end-to-end test walks the same path a real shopper would: Homepage → Login → Product Sorting/Browsing → Add to Cart → Checkout Step One → Checkout Step Two → Order Confirmation, with a Cart Reset check folded in as a final sanity step.

Total Planned Test Cases

Module	Test Case Count
Login	6
Inventory	7
Cart	5
Checkout Step One	3
Checkout Step Two	6
Checkout Complete	3
Total	30

Known Defects Summary

Testing revealed five critical bugs that could disrupt the checkout experience. Full details are in the bug report, with a summary provided in the table below:

Defect ID	Associated Test Case	Module	Defect Summary (Bug Title)	Severity	Priority
BUG-01	TC_ID_14	Cart / Checkout	System allows user to navigate to the checkout page with an empty cart	Critical	High
BUG-02	TC_ID_10	Inventory / Sidebar	'Reset App State' action fails to revert product buttons to 'Add to cart'	Major	High
BUG-03	TC_ID_20	Checkout - Step 1	Zip/Postal Code field lacks validation and accepts invalid text strings	Major	Medium
BUG-04	TC_ID_27	Checkout - Step 2	No flow guard on checkout step 2—accessible directly via URL bypass	Critical	High
BUG-05	TC_ID_38	Checkout - Complete	Checkout Complete page is accessible directly without placing an order	Critical	High

10. Risks & Assumptions

Risks

As with any UI automation suite, the primary risk is locator fragility; if SauceDemo's front-end markup updates, test steps can break even if the core feature functions correctly. Furthermore, slow network responses can cause dynamic elements to load late, potentially causing flaky test results despite utilizing explicit waits. The framework's reliability also heavily depends on stable internet access and the ongoing compatibility between the Google Chrome browser and Selenium WebDriver.

Assumptions

This plan assumes that the SauceDemo application remains online and stable throughout execution, and that the default testing credentials (standard_user / secret_sauce) remain unchanged. It is also assumed that the local machine runs a correctly configured Python 3.14+ runtime with the uv toolchain installed.

11. Deliverables

Upon completion, this automation cycle leaves behind a robust ecosystem rather than just standalone scripts. The primary deliverables include this comprehensive test plan, a reusable Page Object Model (POM) framework architecture, a detailed functional test case repository (docs/test_case_saucedemo.xlsx), an official logged defect sheet (docs/bugreport_saucedemo.pdf), and a comprehensive README.md file detailing quick-start installation and framework execution guidelines.

12. Entry & Exit Criteria

Entry Criteria

Automation execution can begin once the target application is reachable, the local machine is initialized via uv sync to map all requirements, the WebDriver connection to Chrome is verified, and the test scripts are reviewed for execution readiness.

Exit Criteria

The cycle is considered complete once 100% of the planned test cases have run, all discovered defects are documented with attached error logs or screenshots, and the critical end-to-end user flows—from initial login to order confirmation—have been rigorously verified.

13. Conclusion

This test plan establishes a structured, repeatable strategy to validate the SauceDemo application using Selenium WebDriver, Pytest, and POM architecture, managed efficiently via uv. The scope successfully covers the entire e-commerce pipeline. More importantly, by targeting boundary conditions and functional vulnerabilities, the suite has already demonstrated its high-signal value by surfacing five genuine, actionable application defects rather than merely validating the successful standard paths.